DEPLOYGUARD

Cloud & DevOps Interview Preparation Handbook

Self-introduction • Project defense • Core concepts • 20 technical Q&As • 20 scenario Q&As • Calm answering framework

Purpose
A practical handbook for entry-level Cloud/DevOps interviews and DeployGuard/FYP defense. The answers are designed to be technically correct, concise enough to speak in an interview, and connected to real project work.

Prepared for Hassan Sajjad

Cloud Engineering • DevOps • AWS • Terraform • Docker • GitHub Actions

Contents

- 1. Interview mindset and calm answering method
- 2. How to introduce yourself
- 3. How to explain DeployGuard
- 4. DeployGuard goals and architecture talking points
- 5. How to answer when you do not know
- 6. Twenty core Cloud/DevOps questions with answers
- 7. Extra basics to revise
- 8. Twenty full scenario-based questions with model answers
- 9. DeployGuard/FYP defense questions
- 10. Behavioral questions and final revision checklist

Speaking rule

Do not memorize every sentence word-for-word. Memorize the structure and the key technical facts. In an interview, natural understanding is stronger than a rehearsed monologue.

Part I - Interview Mindset and Communication

1. The Calm Answering Framework

A Cloud/DevOps interview is not a speed test. A technically strong candidate can take two or three seconds to organize an answer before speaking. The objective is to show structured reasoning, not to produce the fastest possible response.

Core technical structure
Definition -> Why it matters -> Practical example

For troubleshooting questions, use a different structure:

Scenario structure
Identify -> Investigate -> Fix -> Verify -> Prevent

Example: If asked "What is Docker?", first define it, then explain why teams use it, then connect it to a real deployment workflow. If asked "Why is the website down?", do not jump to a random fix. First identify the failing layer, gather evidence, apply the smallest safe fix, verify recovery, and then explain how you would prevent recurrence.

2. How to Stay Calm During Difficult Questions

- Listen to the entire question before answering. Do not interrupt the interviewer.
- Pause for two to three seconds if needed. This looks deliberate, not weak.
- Restate the problem in one sentence when the scenario is complex.
- Start from first principles: identity, network, compute, storage, application, observability.
- If a term is unfamiliar, say what you do know and avoid inventing details.
- When corrected, accept the distinction and update your answer instead of defending a mistake.
- Keep the first answer around 30-60 seconds unless the interviewer asks for deeper detail.

3. What to Say When You Do Not Know

Safe answer
"I have not implemented that directly yet, but my understanding is ..." Then explain the concept you do understand and, if useful, how you would verify the missing part.

For an entry-level candidate, honest boundaries are better than pretending to have production experience. A strong interviewer is usually evaluating how you think as much as what you have memorized.

4. How to Handle a Correction

Use a calm response such as: "Understood. My understanding was X, but I see the distinction you are making. In that case, I would treat Y as the correct behavior." This shows that you can update your model when new evidence appears.

Part II - Self-Introduction and Project Positioning

5. Recommended 60-90 Second Introduction

Interview introduction

Good morning. My name is Hassan Sajjad. I am a Computer Science student at Air University Islamabad, and my main technical focus is Cloud Engineering and DevOps. My strongest areas are AWS, Linux, networking, Docker, Terraform, GitHub Actions, Git/GitHub, Bash, Python and basic Kubernetes. My major project is DeployGuard, a Platform-as-a-Service that automates application deployment to AWS. It integrates GitHub, GitHub Actions, AWS OIDC, Terraform, ECS Fargate, ECR, Application Load Balancers, Secrets Manager, CloudWatch and managed database infrastructure. Through this project I have worked on deployment automation, IAM, immutable releases, runtime verification, rollback, Terraform state, monitoring and safe infrastructure destruction. I am looking for an entry-level Cloud or DevOps role where I can apply these practical skills and continue learning production engineering practices.

6. Short 30-Second Version

Short introduction

I am a Computer Science student focused on Cloud and DevOps. I work mainly with AWS, Linux, Docker, Terraform and GitHub Actions. My major project is DeployGuard, a PaaS that automates GitHub-to-AWS deployments using OIDC, Terraform, ECS, ECR, ALB and runtime verification. I am looking for a junior Cloud or DevOps role where I can apply this hands-on experience.

7. How to Explain DeployGuard in 45 Seconds

Project explanation

DeployGuard is a Platform-as-a-Service designed to automate application deployment to AWS. A developer connects a GitHub repository, DeployGuard analyzes the application, resolves its runtime requirements, builds it with Railpack or a controlled Docker fallback, pushes an immutable image to ECR, provisions AWS infrastructure using Terraform, and runs the application on ECS Fargate behind an Application Load Balancer. It also supports environment variables, secrets, managed databases, monitoring, redeployment, immutable rollback, runtime verification and safe destruction of project-owned resources.

8. How to Explain the Problem DeployGuard Solves

A developer deploying directly to AWS may need to understand and configure IAM, container registries, ECS, task definitions, networking, load balancers, security groups, secrets, state management, monitoring, and lifecycle operations. DeployGuard turns those separate infrastructure tasks into a controlled deployment workflow while preserving explicit cloud infrastructure and release identity.

9. DeployGuard Goals

Simplify AWS deployment: Automate the repetitive infrastructure and deployment steps required to move application source from GitHub into an AWS runtime.

Make releases reproducible: Associate a release with exact source SHA, immutable image digest, ECS task-definition revision, runtime configuration and deployment operation.

Use infrastructure-as-code: Provision and destroy project resources through Terraform rather than relying only on ad hoc API calls.

Reduce long-lived credentials: Use GitHub OIDC to obtain temporary AWS credentials for GitHub Actions.

Verify runtime reality: Do not declare success only because a workflow completed; verify ECS, ALB, target health, image identity, routing and the public endpoint.

Support the entire lifecycle: Treat Deploy, Redeploy, Rollback and Destroy as one coherent product contract.

Protect configuration and secrets: Use encrypted project environment storage and AWS Secrets Manager/runtime secret references for sensitive values.

Provide operational evidence: Expose logs, metrics, deployment state, failure diagnostics and current runtime identity.

Enable managed persistence: Support managed database runtime paths with persistent storage and controlled network access.

Make cleanup safe: Destroy project-owned resources while preserving shared platform resources and treating already-absent resources idempotently when appropriate.

10. DeployGuard Deployment Flow

1. 1. Developer connects GitHub repository and selects source/branch.
2. 2. Backend validates project configuration and source identity.
3. 3. Deployment admission verifies AWS account, OIDC role/trust and required capabilities.
4. 4. GitHub Actions receives the deployment request.
5. 5. Source is checked out at the exact expected commit.
6. 6. Railpack builds the application; controlled fallback logic is available when required.
7. 7. The image is pushed to Amazon ECR with immutable release identity.
8. 8. Terraform provisions or updates the AWS runtime.
9. 9. ECS Fargate starts the application.
10. 10. ALB/target group routes traffic to the service.
11. 11. Runtime verification checks the actual AWS state and public endpoint.
12. 12. The control plane records the terminal release identity and deployment result.

11. DeployGuard Security Talking Points

- GitHub OIDC and AWS STS temporary credentials for workflow execution instead of storing permanent AWS access keys in GitHub Actions.
- Role trust and AWS capability checks before dispatch.
- RBAC on project operations, with stronger permissions required for mutation actions.
- Authenticated API middleware and separated admin/developer session audiences.

- OAuth state validation for GitHub authentication.
- HTTP-only session cookies, SameSite=Lax and Secure in production.
- HMAC-signed session tokens and expiration checks.
- PBKDF2 password hashing for password-based accounts.
- AES-256-GCM encryption for stored project environment values.
- AWS Secrets Manager references for sensitive runtime data.
- Application security groups accept service traffic from the load balancer rather than opening the application port directly to the internet.
- Database security groups restrict database access to the attached application service.
- Immutable image/task-definition identity for safer rollback and release verification.
- Workflow and executable dependency hash contracts to protect the certified deployment control plane.
- Runtime verification after deployment instead of trusting only CI success.
- Audit logging for important authentication, deployment and administrative operations.
- Log sanitization boundaries to reduce accidental secret leakage.
- Safe destroy confirmation and project-scoped cleanup behavior.

12. How DeployGuard Differs From Vercel, Render and Heroku

Do not present this as "DeployGuard is better." The strongest explanation is that the products have different operating models.

Area	DeployGuard	Vercel	Render	Heroku
Primary operating model	AWS-oriented PaaS/control plane that provisions explicit AWS runtime resources	Highly managed developer platform with strong frontend/serverless/edge abstractions	Managed application/cloud platform with web services, jobs, databases and private networking	Managed application platform centered on dynos/buildpacks and add-ons
Infrastructure visibility	Explicit ECS/ECR/ALB/security-group/Terraform concepts are part of the architecture	Much infrastructure is intentionally abstracted	Provider-managed infrastructure abstraction	Provider-managed infrastructure abstraction
IaC relationship	Terraform is central to project runtime provisioning and destroy	Users can use IaC around Vercel, but deployment platform itself is managed	Infrastructure managed by Render; Blueprint/IaC-style configuration available	Platform configuration and add-ons abstract underlying infrastructure
Release identity	Exact source + immutable image + ECS task-definition/runtime	Deployment/version abstractions managed by	Managed deployment/release model	Release model managed by Heroku

	evidence	Vercel		
Runtime verification focus	Control plane verifies AWS runtime objects and public transport evidence	Platform abstracts most runtime details	Platform exposes managed health/deploy information	Platform exposes app/release/dyno health information
Cloud ownership emphasis	Designed around AWS resources and explicit account/runtime contracts	Vercel-hosted platform model	Render-hosted platform model	Heroku-hosted platform model

The correct interview conclusion: DeployGuard is architecturally closer to an AWS-aware deployment control plane, while the other platforms emphasize a stronger provider-managed abstraction. The distinction is control and infrastructure model, not a simplistic feature winner.

Part III - 20 Core Cloud & DevOps Questions

1. What is cloud computing?

Answer:

Cloud computing is the on-demand delivery of computing resources such as servers, storage, networking, databases and managed services over the internet. It allows organizations to provision capacity quickly instead of purchasing and maintaining all physical infrastructure themselves. The main benefits include elasticity, automation, global availability and usage-based cost models.

Practical example:

DeployGuard provisions AWS services dynamically rather than requiring a developer to manually build a physical or fixed server environment.

2. What is the difference between IaaS, PaaS and SaaS?

Answer:

IaaS provides infrastructure primitives such as virtual machines, networks and storage. PaaS provides a higher-level application deployment platform where much of the infrastructure work is automated. SaaS provides the finished software directly to end users. EC2 is an IaaS example, Heroku/Render are PaaS examples, and Google Workspace is a SaaS example.

Practical example:

DeployGuard is designed as a PaaS/control-plane layer over AWS infrastructure.

3. What is Amazon EC2?

Answer:

EC2 provides resizable virtual machines in AWS. You choose instance type, operating system, storage, networking and security. It gives more server-level control than serverless or fully managed compute services, but that also means more responsibility for patching, scaling and host operations.

Practical example:

A team might use EC2 when it needs host-level control instead of ECS Fargate.

4. What is Amazon ECS?

Answer:

Amazon ECS is AWS container orchestration. It schedules and manages containerized applications using either EC2 capacity or Fargate. ECS services maintain desired task counts, perform deployments and integrate with ALB, IAM, CloudWatch and service discovery.

Practical example:

DeployGuard uses ECS Fargate for application runtime so AWS manages the underlying compute hosts.

5. ECS vs Kubernetes - what is the difference?

Answer:

Both orchestrate containers. ECS is AWS-native and usually simpler for AWS-only environments. Kubernetes is portable and has a larger ecosystem and more abstractions, but it introduces more

operational complexity. The correct choice depends on portability, team skills, ecosystem requirements and operational constraints.

Practical example:

DeployGuard currently benefits from ECS because it integrates directly with ECR, IAM, ALB, CloudWatch and Fargate.

6. What is Docker?

Answer:

Docker packages an application and its dependencies into a container image. A container is an isolated running instance of that image. Containers improve environment consistency across development, CI and production.

Practical example:

DeployGuard builds application images before deploying them into ECS.

7. What is the difference between a Docker image and a container?

Answer:

An image is the immutable package/template. A container is a running instance created from an image. Many containers can be started from the same image, while the image itself does not represent a running process.

Practical example:

ECR stores images; ECS runs containers created from those images.

8. What is Amazon ECR?

Answer:

Amazon ECR is AWS's managed container registry. It stores container images and integrates with IAM and ECS. A typical flow is build -> tag -> authenticate -> push to ECR -> deploy by digest or tag.

Practical example:

DeployGuard pushes release images to ECR and records immutable identity for deployment and rollback.

9. What is Terraform?

Answer:

Terraform is a declarative Infrastructure as Code tool. You describe desired infrastructure in configuration, Terraform compares configuration and state, creates a plan, and then applies required changes through provider APIs.

Practical example:

DeployGuard uses Terraform to create and update ECS, ALB, security groups, IAM runtime roles, EFS and related resources.

10. What is Terraform state?

Answer:

Terraform state maps configuration to real infrastructure and stores attributes required for future planning. Remote state is useful for centralization and team workflows. State is sensitive because it may contain infrastructure metadata and should be protected.

Practical example:

DeployGuard uses environment-scoped S3 state paths for runtime infrastructure.

11. What is Terraform state locking?

Answer:

State locking prevents concurrent writers from changing the same Terraform state at the same time. Without locking, simultaneous applies can conflict and potentially corrupt the relationship between desired and actual infrastructure.

Practical example:

DeployGuard combines scoped state/lock behavior with project-level workflow concurrency controls.

12. What is AWS IAM?

Answer:

AWS IAM controls authentication and authorization for AWS APIs. Major concepts include users, roles, identity policies, resource policies, trust policies and temporary credentials. Effective authorization can also be affected by permission boundaries and organization SCPs.

Practical example:

DeployGuard validates the workflow role and required AWS capabilities before dispatch.

13. IAM user vs IAM role?

Answer:

An IAM user is a long-lived identity and can have persistent access keys. An IAM role is assumed by a trusted principal and normally provides temporary credentials. Roles are preferred for workloads and automation where possible because they reduce reliance on long-lived secrets.

Practical example:

GitHub Actions assumes an AWS role using OIDC rather than storing a permanent AWS access key in the workflow.

14. What is OIDC in CI/CD?

Answer:

OIDC allows a CI/CD system to present a signed identity token to a cloud provider. AWS STS validates the trusted identity and returns temporary AWS credentials if the role trust policy permits the request.

Practical example:

GitHub Actions -> GitHub OIDC token -> AWS STS -> temporary role credentials.

15. What is a security group?

Answer:

A security group is a stateful AWS virtual firewall attached to resources such as ENIs and load balancers. It allows specific inbound and outbound traffic based on protocol, ports and sources. Because it is stateful, return traffic for an allowed connection is automatically permitted.

Practical example:

DeployGuard allows internet traffic to the ALB while the application security group accepts application traffic from the ALB security group.

16. What is an Application Load Balancer?

Answer:

An ALB operates at Layer 7 for HTTP/HTTPS. It distributes requests across targets, performs health checks and supports host/path routing, TLS termination and integration with ECS.

Practical example:

DeployGuard places an ALB in front of ECS application tasks and validates target health during runtime verification.

17. What is a VPC?

Answer:

A VPC is an isolated virtual network in AWS. It contains address ranges, subnets, route tables, gateways, security controls and network interfaces. Resources are placed into subnets and communicate according to routing and security rules.

Practical example:

DeployGuard deploys resources into a configured VPC and subnets.

18. Public vs private subnet?

Answer:

A public subnet has routing that allows direct internet reachability through an Internet Gateway when the resource also has the required public addressing. A private subnet does not provide that direct route. Private workloads typically access the internet through NAT or controlled egress and are reached through internal paths or load balancers.

Practical example:

A production architecture often exposes only the ALB publicly while keeping application and database resources private where possible.

19. What is CI/CD?

Answer:

Continuous Integration automatically validates code through activities such as build, tests, linting and security checks. Continuous Delivery keeps software ready for release; Continuous Deployment automatically releases passing changes. The exact level of automation depends on the organization's risk model.

Practical example:

DeployGuard uses GitHub Actions as part of the controlled build/deploy pipeline.

20. What is GitHub Actions?

Answer:

GitHub Actions is GitHub's workflow automation and CI/CD platform. Workflows contain triggers, jobs and steps and can use repository permissions, secrets, OIDC tokens and reusable workflows.

Practical example:

DeployGuard dispatches a certified reusable workflow that performs the build, AWS authentication, Terraform and runtime verification steps.

Part IV - Extra Basics to Know Before the Interview

Linux basics

- Filesystem hierarchy: /, /etc, /var, /home, /tmp.
- Permissions: read/write/execute, chmod, chown, users and groups.
- Processes: ps, top/htop, kill, signals.
- Services: systemctl, service status, startup and restart.
- Logs: journalctl, /var/log, grep and tail.
- Disk/memory: df, du, free.
- Networking: ip, ss, ping, curl, dig/nslookup, traceroute.
- Remote access: ssh, scp.
- Shell variables, environment variables, pipes and redirection.

Networking basics

- IP address, subnet mask and CIDR.
- Default gateway and routing.
- TCP vs UDP.
- DNS resolution and common record types.
- HTTP vs HTTPS and TLS.
- NAT and Internet Gateway.
- Firewall/security groups.
- Reverse proxy and load balancer.
- Common ports: 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL, 27017 MongoDB, 6379 Redis.

Git basics

- clone, status, add, commit, push, pull and fetch.
- branch, switch/checkout, merge and rebase.
- log, diff, revert and reset.
- Pull requests and code review.
- Merge conflicts and how to resolve them safely.

Bash basics

- Variables and quoting.
- if/elif/else.
- for and while loops.
- Functions.
- Exit codes and `$?`.
- set -e / set -u / pipefail concepts.
- grep, awk, sed, cut, sort, xargs.
- curl and jq for APIs/JSON.

Python basics

- Variables and data types.

- Lists, tuples, sets and dictionaries.
- Functions and modules.
- Loops and comprehensions.
- Exceptions.
- File handling.
- JSON and HTTP APIs.
- Virtual environments and dependencies.

Docker basics

- Dockerfile instructions.
- Build context and .dockerignore.
- Layers and caching.
- Ports and networking.
- Volumes and bind mounts.
- Environment variables.
- Entrypoint vs command.
- Multi-stage builds.
- Registry push/pull.

Kubernetes basics

- Pod, Deployment, ReplicaSet and Service.
- ConfigMap and Secret.
- Ingress.
- Liveness/readiness/startup probes.
- Requests and limits.
- Namespaces.
- Rolling updates and rollback.
- PersistentVolume/PVC basics.

Monitoring basics

- Metrics are numeric time-series values.
- Logs are event records.
- Traces show request paths across components.
- Alerts notify on meaningful conditions.
- Golden signals: latency, traffic, errors and saturation.
- CloudWatch is commonly used for AWS logs/metrics.

HTTP status codes

- 200 OK; 201 Created.
- 301/302 redirect.
- 400 bad request.
- 401 unauthenticated.
- 403 authenticated but forbidden.
- 404 resource not found.
- 500 application/server error.

- 502 bad gateway.
- 503 service unavailable.
- 504 gateway timeout.

Troubleshooting chain to memorize
Code -> Build -> Image -> Registry -> Infrastructure -> Container -> Networking -> Load Balancer -> DNS -> User

Part V - 20 Scenario-Based Cloud & DevOps Questions

Scenario 1 - Website returns 502 behind ALB

Your production website is behind an AWS Application Load Balancer and suddenly returns 502. What do you do?

Model answer:

Identify: Confirm whether the 502 is generated by the ALB, reverse proxy or application.

Investigate: Check ALB target health, ECS service events, task status, container logs, application listen port, security groups and recent deployments.

Fix: Correct the failing layer - for example wrong port, crashed process, unhealthy task or network rule. If the newest release caused the issue, rollback.

Verify: Confirm healthy targets and test the public endpoint from outside the environment.

Prevention: Use health checks, deployment verification, alarms and release rollback capability.

Scenario 2 - ECS task keeps restarting

An ECS task repeatedly starts and stops. How would you troubleshoot it?

Model answer:

Identify: Read the ECS stopped-task reason and container exit code first.

Investigate: Check CloudWatch logs, missing environment variables/secrets, command/entrypoint, CPU/memory limits, architecture compatibility and health checks.

Fix: Resolve the actual cause rather than automatically increasing resources.

Verify: Observe a stable task, healthy service events and successful endpoint response.

Prevention: Add startup validation, clear configuration contracts and monitoring for restart loops.

Scenario 3 - Works locally but fails in ECS

The container works on your laptop but fails in ECS. What differences would you check?

Model answer:

Identify: Assume environment mismatch until evidence shows otherwise.

Investigate: Compare environment variables, secrets, CPU architecture, filesystem assumptions, service dependencies, command/entrypoint, listen address, port and IAM/network access.

Fix: Make the application runtime portable; commonly bind to 0.0.0.0 and use injected configuration rather than localhost-only assumptions.

Verify: Run the same built image with equivalent production configuration before redeploying.

Prevention: Use reproducible images, CI tests and explicit runtime requirements.

Scenario 4 - Terraform apply fails halfway

Terraform created some resources and then failed. What next?

Model answer:

Identify: Read the first failing provider/API error and determine which resources were already created.

Investigate: Use terraform state list and terraform plan rather than deleting everything immediately.

Fix: Correct the specific configuration or permission issue and rerun the plan/apply.

Verify: Ensure plan converges and actual infrastructure matches the desired configuration.

Prevention: Use remote state, locking, reviewable plans and bounded changes.

Scenario 5 - Two engineers run Terraform simultaneously

Two team members trigger Terraform against the same state at the same time.

Model answer:

Identify: Recognize this as a concurrency/state integrity risk.

Investigate: Check whether state locking and CI concurrency are enabled.

Fix: Serialize writes to the same state and release stale locks only after verifying no active operation exists.

Verify: Confirm subsequent plan is consistent and state is healthy.

Prevention: Use remote locking and one writer per state through CI/CD.

Scenario 6 - Bad release reaches production

A new deployment introduces a serious production bug.

Model answer:

Identify: Determine whether the previous release is known healthy and whether data migrations complicate rollback.

Investigate: Compare new vs previous release identity, logs and health evidence.

Fix: Rollback to the previously verified immutable image/task definition if safe.

Verify: Confirm target health, public endpoint and business-critical flows after rollback.

Prevention: Use immutable artifacts, staged deployments, tests and reversible migrations.

Scenario 7 - CPU reaches 95 percent

CloudWatch shows sustained 95% CPU on your service.

Model answer:

Identify: Determine whether the load is expected traffic or abnormal application behavior.

Investigate: Check request rate, latency, logs, hot endpoints, task count, CPU limits and recent releases.

Fix: Scale out/up if workload is legitimate; optimize code or fix loops if it is not.

Verify: Confirm CPU returns to an acceptable range while latency/error rates improve.

Prevention: Configure autoscaling and meaningful CPU/latency/error alarms.

Scenario 8 - Application cannot connect to database

The application is running, but all DB calls fail.

Model answer:

Identify: Separate DNS, network, credential and database-readiness possibilities.

Investigate: Check DB health, hostname resolution, port, security groups, route/network path, username/password, database name, TLS and secret injection.

Fix: Correct the failing configuration or network control.

Verify: Run a real connection/query from the application runtime.

Prevention: Use service discovery, restricted security groups, secret rotation and readiness checks.

Scenario 9 - GitHub Actions cannot access AWS

OIDC-based GitHub Actions fails before Terraform.

Model answer:

Identify: Read the exact STS/AssumeRoleWithWebIdentity error.

Investigate: Verify id-token: write, role ARN, GitHub OIDC provider, audience, trust-policy subject and correct AWS account.

Fix: Correct trust or workflow identity configuration without adding unrelated permissions.

Verify: Run aws sts get-caller-identity inside the workflow and confirm the expected role/account.

Prevention: Keep trust policies explicit and validate account/role before infrastructure operations.

Scenario 10 - AWS AccessDenied

An AWS command returns AccessDenied.

Model answer:

Identify: Record the calling identity, denied action and resource ARN.

Investigate: Check identity policies, resource policies, permission boundaries, SCPs and trust policy if role assumption is involved.

Fix: Add only the required permission at the correct policy layer.

Verify: Retry the exact denied operation and inspect CloudTrail if needed.

Prevention: Use least privilege and pre-deployment capability validation.

Scenario 11 - ALB targets stay unhealthy

ECS tasks are RUNNING but the target group says unhealthy.

Model answer:

Identify: Recognize that RUNNING only means the container process exists, not that the service is ready.

Investigate: Check health-check path/port, application port, security groups, task networking, startup time and response code.

Fix: Correct health check/runtime binding or application startup.

Verify: Wait for healthy targets and then test the application through the ALB.

Prevention: Use proper readiness probes and runtime verification.

Scenario 12 - Docker image is too large

Your image is several gigabytes and deployments are slow.

Model answer:

Identify: Inspect layers and determine what files/dependencies dominate size.

Investigate: Review base image, package caches, build artifacts and build context.

Fix: Use a smaller base image when appropriate, .dockerignore and multi-stage builds; remove build-only dependencies/caches.

Verify: Compare image size and pull/start times after changes.

Prevention: Keep build context small and pin/review dependencies.

Scenario 13 - Secret committed to Git

A production credential was accidentally pushed to a public or shared repository.

Model answer:

Identify: Treat it as compromised immediately.

Investigate: Determine scope, repository history and whether the credential was used.

Fix: Revoke/rotate first, then remove from source/history as appropriate and move it to a secret manager.

Verify: Confirm the old credential no longer works and audit access logs.

Prevention: Use secret scanning, protected secret stores and pre-commit controls.

Scenario 14 - Availability Zone outage

One AWS Availability Zone becomes unavailable.

Model answer:

Identify: Confirm the failure is AZ-specific rather than application-wide.

Investigate: Check service/task placement, subnets, load balancer health and dependent resources.

Fix: Shift/replace capacity into healthy AZs and ensure critical dependencies are also redundant.

Verify: Confirm the load balancer serves healthy targets in remaining AZs.

Prevention: Design stateless services and replicated dependencies across multiple AZs when availability requirements justify it.

Scenario 15 - Pipeline says success but users cannot reach app

CI/CD is green, but the application is unreachable.

Model answer:

Identify: Treat pipeline completion and runtime availability as separate facts.

Investigate: Check ECS task identity, target registration/health, listener, security groups, DNS and public HTTP response.

Fix: Repair the failing runtime/routing layer or rollback.

Verify: Perform an external request and confirm the expected release is serving traffic.

Prevention: Implement post-deploy runtime verification, not workflow-only success.

Scenario 16 - Manual resource deletion causes drift

Someone deletes a Terraform-managed AWS resource through the console.

Model answer:

Identify: Recognize configuration drift.

Investigate: Run terraform plan and inspect actual state/resource existence.

Fix: Allow Terraform to recreate the desired resource or intentionally update configuration/state if the deletion was approved.

Verify: Plan should converge to no unexpected changes after remediation.

Prevention: Restrict manual mutations and use IaC as the normal change path.

Scenario 17 - Need low-downtime redeployment

You need to deploy a new application version without taking all instances down.

Model answer:

Identify: Determine service capacity and readiness requirements.

Investigate: Review deployment strategy, minimum healthy percent, target health checks and startup time.

Fix: Start new tasks, wait until they are healthy, then drain/replace old tasks.

Verify: Monitor errors/latency during the rollout and confirm old targets are safely removed.

Prevention: Use rolling/blue-green strategies, health checks and automated rollback where appropriate.

Scenario 18 - Sensitive data appears in logs

Passwords or tokens are visible in application logs.

Model answer:

Identify: Treat the logs as a potential secret exposure.

Investigate: Determine which values, users and log destinations are affected.

Fix: Stop logging sensitive fields, add redaction/sanitization and rotate exposed credentials when necessary.

Verify: Search new logs to confirm sensitive patterns no longer appear.

Prevention: Use structured logging, allowlists/redaction and restricted log access.

Scenario 19 - Cloud bill suddenly spikes

AWS cost increases sharply after a release.

Model answer:

Identify: Find which service/resource category changed.

Investigate: Compare ECS task count, ALB usage, NAT/data transfer, CloudWatch ingestion, EFS/storage, image retention and scaling events.

Fix: Remove accidental capacity/orphaned resources or correct traffic/architecture issues.

Verify: Track cost and utilization after remediation.

Prevention: Use budgets, cost allocation tags, alerts and FinOps review.

Scenario 20 - Destroy fails halfway

Infrastructure destroy removed some resources but then failed.

Model answer:

Identify: Determine which resources are already absent and which remain.

Investigate: Read Terraform/AWS errors and inspect state before manually deleting anything.

Fix: Retry idempotently: already-absent owned resources should be accepted where safe, but genuine errors such as AccessDenied should still fail.

Verify: Confirm project-owned runtime resources, state objects and locks are removed while shared platform resources remain untouched.

Prevention: Design destroy operations to be retry-safe and ownership-scoped.

Part VI - DeployGuard / FYP Defense Questions

1. Why did you choose ECS instead of Kubernetes?

Answer:

ECS provides the orchestration features DeployGuard needs while reducing operational complexity for an AWS-focused platform. It integrates directly with IAM, ECR, ALB, CloudWatch and Fargate. Kubernetes would become more compelling if multi-cloud portability or Kubernetes ecosystem compatibility became a primary requirement.

2. Why Terraform instead of creating every AWS resource directly through the SDK?

Answer:

Terraform provides a declarative desired-state model, dependency graph, plan/apply/destroy workflow and state tracking. Direct SDK calls are useful for control-plane checks and reconciliation, but Terraform is stronger for reproducible project infrastructure lifecycle.

3. Why GitHub Actions?

Answer:

The application source already lives in GitHub, and reusable GitHub Actions workflows give DeployGuard a controlled execution environment for source checkout, build, AWS OIDC authentication, Terraform and verification.

4. Why OIDC instead of AWS access keys?

Answer:

OIDC reduces reliance on long-lived AWS credentials inside CI. GitHub presents a signed token, AWS STS verifies the trusted subject/audience, and temporary role credentials are issued for the workflow.

5. Why immutable images?

Answer:

Mutable tags such as latest can point to different content over time. Immutable digests provide precise release identity and make rollback/verifiability stronger.

6. Why verify runtime after Terraform?

Answer:

Terraform proves that infrastructure changes were accepted, but it does not prove that the application is actually responding correctly. DeployGuard verifies ECS identity, target registration/health, routing and public transport evidence after provisioning.

7. Why keep deployment generations?

Answer:

A generation groups one coherent deployment state: source identity, service revisions, task definitions, routes, runtime bindings and release metadata. It reduces ambiguity when reconciling current state and rollback history.

8. Why rollback without rebuilding old source?

Answer:

A rebuild can produce a different artifact because dependencies, base images or registries may have changed. Restoring the prior immutable image/task definition gives a stronger guarantee that the exact previously verified artifact is restored.

9. How do you protect application environment variables?

Answer:

Project environment values are encrypted at rest by the control plane using AES-256-GCM derived from the session secret, and sensitive runtime values are materialized through AWS Secrets Manager references rather than being treated as plain application source.

10. How is database traffic restricted?

Answer:

The database runtime security group allows the database port from the attached application security group rather than from the public internet. Persistent database storage uses EFS with its own NFS security-group boundary.

11. What makes destroy safe?

Answer:

Destroy is project-scoped, uses Terraform for managed infrastructure, verifies cleanup evidence, preserves shared platform resources, and treats specific already-absent resources as successful idempotent cleanup while still failing on real permission or API errors.

12. How do you know which release is actually live?

Answer:

The platform records exact source SHA, image identity and ECS task-definition/runtime evidence, and it reconciles this with AWS during runtime verification/current-state logic rather than relying only on the database record.

13. What was one major defect you fixed?

Answer:

One class of defect involved dynamic `$PORT` handling. A parser could reject `$PORT` as a non-numeric port even though frameworks commonly use it. The contract was changed so dynamic PORT references are accepted and framework/runtime defaults can resolve correctly.

14. What was one IAM-related lesson?

Answer:

Permissions must be modeled against the exact APIs and exact resource paths used by the workflow. For example, state-resource simulation must include the environment-scoped key, and managed EFS mount targets have dependent EC2 network-interface API requirements.

15. How do you separate application failure from platform failure?

Answer:

The deployment pipeline records stage-specific evidence and runtime diagnostics. I troubleshoot in layers: source/build -> image -> Terraform -> AWS infrastructure -> container/runtime -> load

balancer/network -> public endpoint. This avoids blaming AWS for an application startup failure or vice versa.

Part VII - Behavioral Questions

1. Tell me about a difficult problem you solved.

Answer:

A strong example is runtime detection and deployment compatibility. Different frameworks expose startup commands and ports differently. I separated detection, validation and runtime verification rather than relying on one assumption. One concrete issue was `$PORT` handling, which was corrected through an explicit contract and verification tests.

2. Tell me about a failure.

Answer:

During development, some failures that looked like AWS issues were actually application startup or configuration problems. I learned to diagnose layer-by-layer using concrete evidence from the source, build, container, Terraform, ECS, load balancer and endpoint rather than changing infrastructure randomly.

3. What is your weakness?

Answer:

I have stronger practical experience with AWS ECS/Terraform than with large production Kubernetes environments. I understand Kubernetes fundamentals, but I do not claim that I have operated a large production Kubernetes cluster. I am continuing to build that experience.

4. Why should we hire you?

Answer:

I am applying at junior level, so I do not claim senior production experience. What I offer is practical exposure beyond tutorials: AWS infrastructure, containers, Terraform, GitHub Actions, IAM/OIDC, load balancing, monitoring and failure diagnosis through DeployGuard. I can contribute with these fundamentals while learning production practices from experienced engineers.

5. How do you learn a new technology?

Answer:

I first learn the core model and terminology, then build a small working implementation, deliberately break it, diagnose failures, document the result and finally integrate it into a larger project. I prefer evidence from official documentation and real behavior over memorizing commands.

Part VIII - Final Revision Checklist

- Explain your 60-90 second introduction without reading.
- Explain DeployGuard in 45 seconds and in 2 minutes.
- Draw the deployment path from GitHub to GitHub Actions to OIDC to AWS to ECS/ALB.
- Explain Docker image vs container.
- Explain ECS vs EC2 vs Kubernetes.

- Explain Terraform plan/apply/state/locking/destroy.
- Explain IAM user vs role and OIDC trust.
- Explain VPC/subnet/security group/ALB/DNS basics.
- Explain 401 vs 403, 502 vs 503 vs 504.
- Explain why an ECS RUNNING task may still be unhealthy.
- Explain immutable deployment and rollback.
- Explain how you troubleshoot AccessDenied.
- Explain how you troubleshoot application -> database connectivity.
- Explain Linux processes, services, logs and permissions.
- Write a small Bash loop/if/function and explain exit codes.
- Explain Git branching, merge, rebase, revert and reset.
- Explain metrics, logs, traces and alerts.
- Explain basic Kubernetes Pod/Deployment/Service/Ingress concepts.
- Practice at least five scenario answers aloud using Identify -> Investigate -> Fix -> Verify -> Prevent.
- Never claim production-scale experience you do not actually have.

Final interview rule
Your strongest answer is not the longest answer. It is the answer that is technically correct, structured, honest about boundaries, and connected to evidence from real work.

One-page mental model

For most Cloud/DevOps troubleshooting questions, mentally walk through:

End-to-end chain
Identity -> Source -> Build -> Image -> Registry -> IaC -> Cloud resources -> Container -> Service -> Load balancer -> DNS -> User -> Logs/Metrics

If you can identify where this chain first stops matching the expected state, you can usually narrow the problem quickly and answer the scenario in a disciplined way.